

Cuprobe: Precise and Efficient GPU Profiling with eBPF-enabled Instrumentation

Anonymous Author(s)

Submission Id: 190

Abstract

Precise GPU observability and programmability are essential for optimizing performance in AI workloads and other computationally intensive high-performance computing (HPC) applications. In this paper, we introduce Cuprobe, the first framework and eBPF runtime that dynamically offloads eBPF bytecode onto GPUs via dynamic PTX injection. Designed primarily for observability, our system leverages real-time GPU telemetry, eBPF-based dynamic instrumentation, and automated performance analysis to pinpoint bottlenecks at multiple layers—including kernel execution, memory transfers, and heterogeneous compute orchestration—without incurring significant overhead or interrupting active GPU kernels. By dynamically compiling eBPF programs into PTX snippets and injecting them directly into running GPU kernels, Cuprobe provides fine-grained, low-overhead instrumentation that seamlessly integrates with existing eBPF ecosystems. We demonstrate Cuprobe’s versatility through three representative use cases: MemTrace, where on-the-fly GPU memory-access tracing identifies cache-miss patterns and bank conflicts; Fair Share Scheduling, which injects instrumentation to analyze and adapt fair share scheduling in order for improved load balancing; CPU–GPU Collaborative Inference, enabling coordinated profiling across host and device to optimize data-transfer pipelines for ML Green use.

1 Introduction

Developers are increasingly tasked with modifying and extending operating system kernels to improve performance, security, and reliability, or introducing new features to their systems. Extended Berkeley Packet Filter (eBPF) have emerged as the de facto method for extending an operating system, with recent support for both Linux and Windows [8]. eBPF programs inject new logic that is executed before or after existing kernel logic to observe or modify the kernel’s behavior. eBPF programs were originally used to trace network traffic, but the ecosystem now provides sufficient power to implement a variety of features, including performance monitoring performance [11, 12], intrusion detection [3, 18], and application-specific logic [10, 19, 42, 43]. eBPF has emerged as a powerful observability framework, traditionally employed in network analytics [4, 5, 10, 41, 46], system verification [32, 40], system performance[28], system security [20, 39], and system profiling [42, 44]. eBPF allows lightweight, sandboxed instrumentation programs to execute in kernel space without intrusive kernel changes or

performance penalties. However, existing eBPF techniques remain CPU-centric, constrained by frequent kernel-user space context switches and limited visibility into the GPU execution model.

The rapid growth of data-intensive and computationally demanding workloads—ranging from large-scale machine learning and high-performance computing (HPC) simulations [7, 9, 24, 26, 34, 36, 45] to graphics rendering [30, 33] and large language models [13, 25]—has increasingly relied on GPU acceleration to achieve desired performance and throughput. As these GPU-powered systems scale, precise observability and low-overhead instrumentation become essential for diagnosing performance bottlenecks, optimizing resource usage[16, 23], and identifying anomalous behavior at runtime. Existing GPU instrumentation approaches, such as NVIDIA’s CUDA Profiling Tools Interface (CUPTI) [29] and binary-instrumentation frameworks like NVBit [38], offer insights into GPU performance but often incur substantial overhead or require invasive modifications that interrupt GPU kernels.

Applying eBPF to GPU workloads presents unique challenges: GPU execution models feature asynchronous kernel launches, parallel thread execution, and specialized memory hierarchies, making instrumentation significantly more complex and prone to high overhead. Furthermore, eBPF’s inherent CPU-centric design does not directly extend to GPU hardware without novel translation and runtime adaptation strategies.

To bridge this gap, we introduce Cuprobe, the first observability framework and eBPF runtime that dynamically extends eBPF instrumentation into GPU kernels via runtime PTX injection. Specifically, Cuprobe compiles eBPF bytecode into NVIDIA’s Parallel Thread Execution (PTX) intermediate representation at runtime and injects this instrumentation directly into actively executing GPU kernels. Leveraging user-space eBPF uprobes, shared-memory maps, and real-time GPU telemetry, Cuprobe provides fine-grained GPU instrumentation with minimal runtime overhead. Unlike previous GPU profiling solutions, our approach allows instrumentation to be dynamically added, modified, or removed when running GPU kernels without interrupting their execution.

Moreover, while initially demonstrated through precise GPU observability, the broader significance of dynamically injecting eBPF onto GPUs extends beyond instrumentation alone. Our methodology opens a window for programmable

GPU computing, potentially enabling adaptive runtime kernel optimizations, dynamic GPU resource management, real-time security policy enforcement, and other novel runtime GPU modifications previously inaccessible to existing static or high-overhead instrumentation approaches.

In summary, we make the following core contributions:

- We introduce Cuprobe, the first system to dynamically offload eBPF instrumentation and bytecode directly onto running GPU kernels using real-time PTX injection, significantly reducing instrumentation overhead compared to existing methods.
- We detail the design and implementation of Cuprobe, which integrates kernel-level and user-space eBPF instrumentation hooks, runtime PTX generation, and shared-memory synchronization, providing a seamless, low-overhead observability platform for modern HPC and AI workloads.
- We evaluate Cuprobe through initial micro-benchmark experiments measuring GPU memory access latency instrumentation, demonstrating practical low overhead and high-resolution insights into GPU execution.
- We discuss broader implications and future possibilities enabled by Cuprobe, highlighting its potential as a foundational infrastructure for three usecases, MemTrace, fair share scheduling, and ML Green.

2 Background

In this section, we will introduce the foundational concepts that underpin our discussion in the subsequent sections.

2.1 User Space Application Extension

In recent years, the eBPF ecosystem has grown well beyond its original role as a kernel-level packet filter. Today eBPF serves as a unified infrastructure for observability, security enforcement, and programmable networking, and it integrates tightly with modern development workflows via tools like clang, libbpf and BTF-based CO-RE. Although traditional eBPF programs run in kernel space for direct access to low-level subsystems, this arrangement can introduce friction when instrumenting user-level code—each uprobe (user-level probe) still incurs a full context switch into the kernel, along with the overhead of saving and restoring CPU state. To address these latency and permission barriers, as shown in Figure 1, recent efforts have begun to push eBPF functionality into user space. By relocating probes and map operations into the process itself—often via binary rewriting or in-process hooks—developers can avoid costly kernel crossings, reduce reliance on elevated privileges, and achieve finer-grained configurability. User-space eBPF extensions also simplify multi-process coordination through shared-memory map implementations and allow seamless integration with existing eBPF toolchains, giving developers the flexibility to deploy sophisticated observability and security logic without kernel recompiles or system restarts.

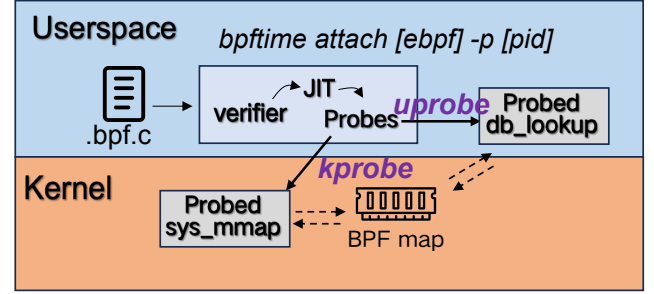


Figure 1. Userspace BPF Diagram

2.2 PTX Just In Time Compilation

PTX (Parallel Thread Execution) serves as an intermediate representation for NVIDIA GPUs, offering a device-independent layer for parallel computations, which is faster than NVBit[21, 38] tools. PTX Just-In-Time (JIT), used in [14, 17] compilation allows developers to convert PTX code into GPU-specific machine instructions on the fly, enabling them to adapt and optimize their programs in real-time. This capability extends beyond simple runtime compilation: it facilitates dynamic instrumentation by letting users intercept and modify PTX instructions before they are turned into final machine code (SASS). In doing so, developers can embed profiling counters, memory checks, or custom analytics directly into the GPU kernel with minimal overhead. Because PTX JIT operates at this higher-level IR—rather than at the raw binary level as in frameworks like NVBit—it often delivers superior performance, as it bypasses the complexity of rewriting fully compiled machine instructions. This leads to highly optimized code tailored to the specific GPU architecture, capitalizing on features such as register usage and warp scheduling. Additionally, PTX JIT integrates naturally with existing NVIDIA toolchains (including NVCC and NVRTC), letting users refine or generate specialized kernels without leaving familiar workflows, and removing the need for time-consuming offline recompilation every time a kernel needs to change or be instrumented.

2.3 GPU Scheduler and Secure Sandbox

Beyond CPU-based Confidential Virtual Machines, confidential computing for GPUs has emerged to protect large-scale model workloads during both inference and training. Soter [35], an edge-oriented design, relies on a CPU-side enclave and parameter morphing to offload selected layers to the GPU while keeping model weights secret. In practice, however, Soter incurs a $2\times-15\times$ latency penalty on LLM inference because non-associative components (e.g. token embeddings) cannot be moved off the CPU. Honeycomb [27] takes a different tack, inserting a trusted software layer and performing static validation of every indirect memory access. While this blocks unsafe GPU commands—an issue exploited in vLLM’s request scheduler [25]—the added runtime checks

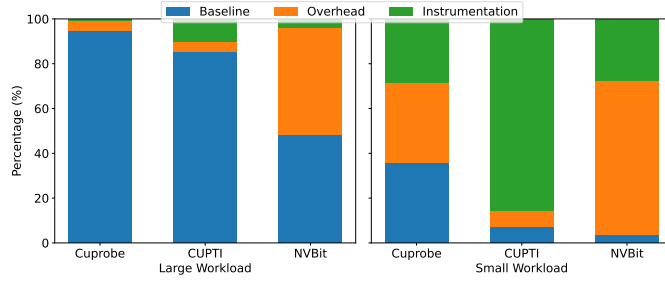


Figure 2. Null Tool Runtime and Instrumentation Overhead Breakdown

impose substantial performance overhead. Han *et al.* [15] further explore microsecond-scale scheduling for GPU, but their kernel-level modifications introduce non-trivial latency and reduce compatibility. In contrast, PipeLLM[37] achieves secure, low-overhead GPU scheduling by speculatively pipelining AES-GCM encryption with data transfers. Without any changes to application code or hardware, PipeLLM overlaps encryption, copy, and compute—eliminating up to 80 % of the confidentiality overhead and maintaining end-to-end latency increases below 20 %.

2.4 Experiment Observation

Existing GPU instrumentation approaches, such as NVIDIA’s CUDA Profiling Tools Interface (CUPTI) [29] and binary-instrumentation frameworks like NVBit [38], offer valuable insights into GPU performance but often incur substantial overhead or require invasive modifications that interrupt GPU kernels. Meanwhile, Linux’s extended Berkeley Packet Filter (eBPF) has emerged as a powerful, lightweight observability framework—traditionally employed in network analytics [10] and system profiling [42, 44]—that allows sandboxed instrumentation programs to execute in kernel space without intrusive changes or significant performance penalties. However, existing eBPF techniques remain strictly CPU-centric, constrained by frequent kernel–user-space context switches, and offer limited API-level visibility into the GPU execution model.

Modern GPU workloads, however, demand real-time, fine-grained instrumentation that can adapt on the fly without halting long-running kernels. We identify three representative use cases that highlight these requirements:

As shown in Figure 5, neither CUPTI nor NVBit can inject custom instrumentation logic directly into running kernels, and CPU-only eBPF solutions cannot observe or control GPU-specific behaviors without stopping or relaunching kernels. To bridge this gap, we introduce Cuprobe, the first framework and eBPF runtime that dynamically extends eBPF instrumentation into GPU kernels via runtime PTX injection. Specifically, Cuprobe:

- Compiles eBPF bytecode into NVIDIA’s Parallel Thread Execution (PTX) intermediate representation at runtime.
- Injects PTX snippets directly into actively executing GPU kernels without interrupting them.
- Leverages user-space eBPF uprobes, shared-memory maps, and real-time GPU telemetry to provide fine-grained, low-overhead instrumentation.

By enabling on-the-fly addition, modification, or removal of instrumentation logic, Cuprobe empowers new classes of dynamic GPU observability and programmability, satisfying the stringent requirements of MemTrace, Block Scheduling, and CPU–GPU Collaborative Inference use cases without compromising kernel continuity or performance.

3 Cuprobe Design

3.1 Cuprobe and Curetprobe

Cuprobe provides a dynamic, eBPF-based “uprobe/uretprobe” mechanism for GPU functions by attaching instrumentation at user-land function entry and exit points. For example, the following eBPF program traces a tiled matrix-multiply kernel (matMulTiled), incrementing a call counter and measuring total execution time:

```

1 SEC("cuprobe/_Z11matMulTiledPKfS0_Pf")
2 int cuprobe_matMulTiled(struct pt_regs *ctx) {
3     u32 pid = bpf_get_current_pid_tgid() >> 32;
4     u64 ts = bpf_ktime_get_ns();
5     bpf_map_update_elem(&start_ts, &pid, BPF_ANY);
6     u64 one = 1, *cnt = bpf_map_lookup_elem(&call_count,
7     &pid); // increment call count
8     if (cnt) *cnt += 1;
9     else bpf_map_update_elem(&call_count, &pid, &one,
10     BPF_ANY);
11 }
12 SEC("curetprobe/_Z11matMulTiledPKfS0_Pf")
13 int curetprobe_matMulTiled(struct pt_regs *ctx) {
14     u32 pid = bpf_get_current_pid_tgid() >> 32;
15     u64 *tsp = bpf_map_lookup_elem(&start_ts, &pid);
16     if (!tsp) return 0;
17     u64 delta = bpf_ktime_get_ns() - *tsp;
18     bpf_map_delete_elem(&start_ts, &pid);
19     u64 *total = bpf_map_lookup_elem(&total_time_ns,
20     &pid); // accumulate execution time
21     if (total) {
22         *total += delta;
23         bpf_map_update_elem(&total_time_ns, &pid,
24         total, BPF_EXIST);
25     } else {
26         bpf_map_update_elem(&total_time_ns, &pid,
27         &delta, BPF_NOEXIST);
28     }
29 }

```

Curetprobe extends this model onto the GPU by translating these same eBPF probe handlers into PTX instrumentation snippets and injecting them directly into running CUDA

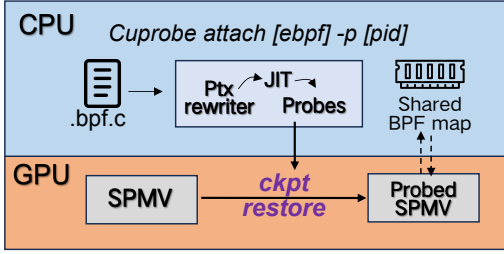


Figure 3. Cuprobe Design

kernels. At JIT time, Cuprobe compiles the above eBPF bytecode into equivalent PTX trampolines at the entry and exit of matMultiled within the GPU kernel image. These trampolines record per-warp timestamps and update shared eBPF maps exactly as their CPU counterparts. By preserving the original map semantics, Curetprobe achieves function-level profiling on the device without kernel reloads, delivering sub-microsecond instrumentation overhead and live visibility into GPU execution.

3.2 Userspace eBPF integration

Figure 3 depicts our unified user-space eBPF architecture, which seamlessly integrates traditional eBPF toolchains with GPU-resident instrumentation. Starting on the left, eBPF programs are compiled from source using familiar tools such as clang, bpftool, or bpftrace into standard eBPF bytecode. Rather than loading directly into the kernel, this bytecode is handed off to a user-space runtime (e.g., via libbpf) that includes a user-space verifier and JIT compiler (bpftime-syscall.so), enabling safety checks and transformations without requiring elevated privileges. When necessary, the same runtime may invoke the bpf() syscall to deploy programs or maps into the kernel subsystem, but by default, all instrumentation remains in user space. At the core of the design is a shared-memory region—allocated, for instance, with boost::managed_shared_memory—that hosts eBPF maps accessible to both CPU threads and GPU kernels with zero copy. On the right, the JIT compiler can translate eBPF bytecode into PTX snippets and inject them into running CUDA kernels at designated hook points (akin to uprobes), allowing GPU-side probes to record events directly into the same eBPF maps. This arrangement affords flexible attach targets—spanning kernel probes, user-level functions, or GPU hooks—and provides real-time, low-overhead observability across CPU and GPU domains while preserving compatibility with existing eBPF workflows.

3.3 Cuprobe Gate

Figure 4 illustrates how we safely extend a user-space eBPF runtime into GPU kernels while enforcing strict isolation and coordinated shared memory. On the CPU side (left), the eBPF runtime executes inside an MPK-protected domain, ensuring

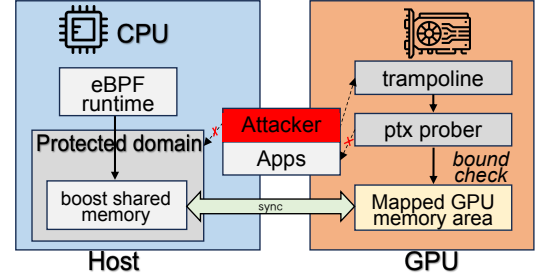


Figure 4. Cuprobe Gate design for CPU-GPU communication, showing the MPK+CFI-protected shared memory region, trampoline hooks, and PTX injector within the isolation boundary.

that only vetted instrumentation code can write into the shared region. All program loads and updates happen under MPK and CFI enforcement. Crossing the isolation boundary (shaded blue), the MPK Gate module mediates every CPU and GPU syncing interaction: any request to modify instrumentation state in shared memory must pass through the Gate, which validates MPK permissions before allowing the write.

The shared memory region (center), implemented with boost::managed_shared_memory, is mapped into both CPU and GPU address spaces. This single region lets updates from the eBPF runtime appear immediately on the device without extra copies. On the GPU side (right), we launch a long-running “persistent” kernel that includes small trampoline stubs at each hook point. When execution reaches a stub, the kernel atomically jumps into the PTX Injector, which fetches and executes the latest instrumentation snippet (compiled from your eBPF bytecode) directly from shared memory. Upon completion, control returns to the original kernel logic.

To guarantee safety, we apply static GPU memory-bound-check analysis on all injected code. This ensures that no instrumentation can stray into GPU-private buffers or access CPU memory outside the MPK-protected region. By keeping the kernel alive and dynamically swapping in new PTX probes, we eliminate repeated launches and enable truly live, on-the-fly GPU observability.

3.4 GPU Trampoline

To support in-place instrumentation without tearing down long-running GPU kernels, Cuprobe employs a lightweight PTX-level “trampoline” mechanism. A trampoline is a small stub of injected PTX code inserted at a designated patch point—typically at the entry or exit of a CUDA kernel function or at key memory-access sites. When the GPU’s program counter reaches this stub, execution temporarily diverts into our instrumentation logic before resuming the original kernel flow.

Concretely, each trampoline stub performs three steps:

- **Context Save:** Push the minimal set of live registers and predicate flags into a per-warp scratch buffer in shared memory, preserving the kernel's architectural state.
- **Instrumentation Jump:** Branch into the injected PTX snippet—compiled on the fly from eBPF bytecode—which may read or update shared eBPF maps, record timestamps, count events, or apply custom analysis.
- **Context Restore:** Reload the saved registers and flags, then jump back to the next instruction in the original kernel code.

By generating trampolines at JIT time and patching branch targets atomically, Cuprobe ensures that no global kernel relaunch or module reload is required. Moreover, trampolines are confined within the *protected domain* (see Section 4.6), so they may only access approved shared memory regions and may not overwrite application buffers. In our experiments, the added latency per trampoline invocation remains under 1 μ s, enabling fine-grained, live instrumentation of GPU kernels with negligible impact on overall throughput.

4 Cuprobe Implementation

In this section, we will talk about the Cuprobe implementation.

4.1 Self-Modifying Code

We require true self-modifying PTX on the device to stream new instrumentation into a long-running kernel, but current NVIDIA drivers offer no direct support. To work around this, we piggy-back on ParallelGPU OS (POS) [17], an OS-level framework that transparently checkpoints and restores GPU applications without any source changes. POS hooks every CUDA driver call to build a fine-grained DAG of kernel launches and their buffer read/write sets. By speculatively inferring each kernel's buffer accesses from its arguments—and then validating those patterns at runtime—POS provides “soft” copy-on-write, dirty-bit, and on-demand-load semantics on hardware that lacks those primitives natively.

Building on POS's buffering and dependency infrastructure, our runtime emits JIT-compiled PTX fragments and patches them directly into the resident kernel image. Whenever a new PTX snippet is injected, POS records the new code region in its DAG, applies soft copy-on-write to any overlapping buffer pages, and validates that the modified code and data views remain consistent before letting the kernel proceed.

This tight integration allows us to perform live code transformations—such as installing trampoline stubs or altering kernel logic—without restarting the GPU workload or risking state corruption. Thanks to POS's dependency-driven scheduler and lightweight buffer-tracking, we achieve: (1)

Transparent correctness: POS's emulated COW and dirty-bit mechanisms isolate code patches and data updates, ensuring semantic integrity. (2) Concurrent operation: Checkpoint/restore tasks run in parallel with instrumentation injections, minimizing interruptions. Low latency: By avoiding full memory copies, self-modification incurs under 20 μ s per patch on average. Overall, POS provides a robust substrate for on-the-fly PTX self-modification, enabling production-grade dynamic instrumentation of GPU kernels.

4.2 Shared Memory

In our design, we leverage boost::managed_shared_memory to create a unified buffer that both the host and device can access without expensive copies. This shared region holds critical data structures—eBPF maps, input batches, and intermediate states—mapped simultaneously into CPU and GPU address spaces. By ensuring GPU-compatibility at allocation time, we eliminate the need to marshal or memcpy data between domains, yielding lower latency and reduced code complexity. Emerging architectures such as NVIDIA's Grace Hopper promise hardware-coherent, large-scale memory pools over high-speed interconnects. On such platforms, our managed_shared_memory can transparently scale to multi-gigabyte working sets, with fine-grained coherence handled by the interconnect fabric. This advancement will further minimize synchronization overhead and allow kernels to process broader data ranges without the usual replication or barrier penalties.

4.3 Cross-Domain Memory Synchronization

On discrete CPU-GPU platforms without hardware cache coherence, every shared-memory access across domains must incur explicit barriers and synchronization, which often translates into tens to hundreds of microseconds of overhead. To transfer writes from the GPU to the CPU, the device must execute __threadfence_system() to flush any outstanding writes into system-visible memory, followed by a host-side cudaStreamSynchronize() to wait for the GPU to complete. Conversely, after the CPU updates shared memory, it must again synchronize with the GPU (via cudaStreamSynchronize() or a recorded cudaEvent) before the device can read the new values reliably. Each such round trip involves driver round-trips, PCIe or NVLink transactions, and potential context switches, all contributing to substantial latency.

In our implementation, we encapsulate these low-level operations into a concise PTX instruction called memsys_bar. The device-side function memsys_bar_gpu() invokes __threadfence_system() to ensure that all prior writes to host-pinned memory are visible system-wide. On the host side, memsys_bar_cpu() calls cudaStreamSynchronize() on a specified stream to wait for the GPU to finish its pending commands, including memory writes. This wrapper simplifies code, clarifies intent, and centralizes performance tuning.

```

1 // Ensure GPU writes become visible to the CPU
2 __device__ inline void memsys_bar_gpu() {
3     __threadfence_system();
4 }
5 // Wait for the GPU on the given stream
6 static inline void memsys_bar_cpu(cudaStream_t stream =
7     0) {
8     cudaStreamSynchronize(stream);
9 }

```

To demonstrate usage, consider a persistent kernel that reads a command flag from shared memory. Before loading the flag, the kernel calls `memsys_bar_gpu()` to enforce visibility of CPU-side writes. The host code updates the flag and then calls `memsys_bar_cpu()` to guarantee the GPU will observe the change when it next schedules work. This two-line barrier abstraction hides the underlying complexity of cross-domain synchronization.

```

1 using namespace boost::interprocess;
2 __device__ int *shared_flag;
3 __global__ void persistent_kernel() {
4     memsys_bar_gpu(); // Ensure CPU writes are visible
5     int cmd = *shared_flag;
6     // Jump into PTX injector...
7 }
8 int main() {
9     managed_shared_memory shm(open_or_create, "MyShm",
10        , 1 << 20);
11     int *flag = shm.construct<int>("Flag")(0)[1];
12     *flag = 42; // Write command
13     memsys_bar_cpu(0); // Ensure GPU sees update
14     persistent_kernel<<<grid, block>>>();
15     cudaDeviceSynchronize();
16 }

```

Looking into systems like NVIDIA's Grace Hopper, which support hardware cache coherence across CPU and GPU, these explicit barriers will become unnecessary. Under a coherent interconnect such as NVLink/CXL, cache lines are automatically propagated and merged at the L2/L3 levels, eliminating the need for manual `__threadfence_system()` or stream synchronizations. Early measurements indicate that cross-domain latency on such platforms can shrink from tens of microseconds to just hundreds of nanoseconds, reducing synchronization overhead by an order of magnitude and enabling truly low-latency shared-memory coordination.

4.4 Synchronization

Given that multiple CPU threads and GPU kernels share data structures, synchronization must be carefully managed to avoid race conditions and ensure memory consistency. We rely on several mechanisms that operate at different levels. On the CPU side, standard lock-based or lock-free data structures (e.g., using `std::atomic` or Boost `interprocess`

primitives) ensure that host processes do not introduce inconsistent states in shared regions. Within the GPU, we use CUDA atomic operations and device-wide memory fences to serialize updates to global or shared memory. This combined strategy, supported by a well-defined data layout in `managed_shared_memory`, ensures that asynchronous updates remain visible to all participants. For frequent small-scale synchronization events, we minimize warp-level conflicts by having only a small subset of threads perform atomic operations, whereas the rest of the kernel continues to process data in parallel.

4.5 PTX Generation & Injection

Our system performs eBPF or application-specific code transformations into PTX to enable GPU-side instrumentation and computation. When user-space or eBPF logic requests that certain functions be offloaded or traced on the GPU, our JIT compiler generates specialized PTX snippets. These are then dynamically injected into the running kernel through device-resident hooks. We refer to this technique as "PTX injection," which allows us to intercept or insert instructions without tearing down the existing kernel context. This approach helps reduce overhead compared to stopping the kernel, recompiling the entire codebase, and relaunching it.

Once injected, the PTX code can interoperate with eBPF maps or other shared memory buffers, granting it direct access to the same data structures used by the CPU side. PTX injection creates a flexible environment for GPU instrumentation, tracepoints, and user-defined logic that can adapt in real time as new workloads arrive or instrumentation needs change. By carefully managing memory barriers and code patching points, we ensure that newly injected PTX interacts cleanly with the previously running kernel components.

Trampoline Mechanism. To insert instrumentation at arbitrary points within a live kernel, Cuprobe employs a PTX-level *trampoline* mechanism. At runtime, we identify candidate patch points (e.g., function entry, memory-access sites, or barrier syncs) and emit a small trampoline stub in PTX that:

- Saves the minimal GPU register and predicate state.
- Jumps into the injected instrumentation snippet.
- Restores the original state and resumes normal execution.

These trampolines are linked into the device's code image via in-place patching of branch targets, enabling fine-grained hooking without requiring kernel relaunch or full recompilation. Our trampoline generator handles label resolution, register allocation, and insertion of necessary synchronization primitives to avoid hazards.

4.6 Isolation Requirement

Dynamic PTX injection and trampoline-based instrumentation introduce new code into live GPU kernels, which must be prevented from corrupting application state or violating

memory safety. To ensure that injected instrumentation runs in a strictly controlled environment, Cuprobe enforces an *isolation requirement* by placing all injected snippets into a protected execution domain.

Guardian-inspired GPU Partition. Drawing on the principles of Guardian’s PTX-level bounds checking for multi-tenant GPU sharing [31], Cuprobe partitions the GPU address space at runtime and instruments every load/store with lightweight fencing logic:

- **Dynamic Partitioning:** At initialization, Cuprobe uses one partition of entire GPU memory. Each instrumentation snippet is tagged with its owner’s partition ID.
- **PTX-Level Bounds Fencing:** Before every memory access, bitwise AND/OR operations mask the computed address into the tenant’s partition range, guaranteeing that out-of-bounds addresses wrap harmlessly within the correct domain.
- **Client–Server Enforcement:** Inspired by Guardian’s client–server model, all CUDA calls (allocations, copies, launches) are intercepted and forwarded to a trusted Manager. This server maintains the partition bounds table and rejects any operation that would violate domain isolation.

Runtime Sandbox Integration. During JIT compilation, Cuprobe wraps each PTX snippet in a minimal prologue/epilogue sequence that: a) Saves a live register state to a safe context. b) Applies bitwise partition masking and invokes the instrumentation logic. c) Restores registers and resumes normal kernel execution.

By combining Guardian’s dynamic, PTX-level fencing with Cuprobe’s trampoline-based patching and shared-memory synchronization, we ensure complete isolation of instrumentation code, preventing any accidental or malicious interference with application kernels.

5 Use Case

5.1 GPU Memory Tracer

The GPU Memory Tracer use case demonstrates how Cuprobe can provide fine-grained, real-time insight into memory access behavior inside running kernels. By instrumenting the core load (LD) and store (ST) instructions in GPU code—via PTX trampolines injected at compile time—our tracer records each memory operation’s address, size, warp ID, SM ID, and timestamp directly into a shared eBPF map. In practice, a developer enables MemTrace by specifying the target kernel and mapping expression; at runtime, Cuprobe patches in lightweight PTX stubs that log metadata into a shared memory region. A user-space analysis callback then consumes these events with sub-microsecond latency, computing metrics such as instantaneous bandwidth

$$BW(t) = \frac{\sum_{i:t_i \in [t-\Delta, t]} \text{Bytes}_i}{\Delta},$$

as well as coalescing ratios and per-SM contention levels. This capability proves invaluable when tuning memory-bound kernels—developers can, for example, spot unexpected bank conflicts or uncoalesced accesses in a tiled convolution, experiment with varying tile sizes, and immediately observe the impact on conflict rates and throughput without restarting the application.

5.2 Fair Share Scheduling

The Fair Share Scheduling use case leverages Cuprobe’s block-level instrumentation to achieve proportional resource allocation among concurrently running GPU workloads. In a shared cluster environment, multiple tenants may submit kernels with heterogeneous characteristics—some compute-bound, others memory-bound, and still others exhibiting irregular control flow. To prevent any single tenant from monopolizing SM occupancy or memory bandwidth, Cuprobe injects PTX trampolines at each CUDA block’s entry and exit, logging per-block execution durations into the same shared eBPF map used by MemTrace. A host-side scheduler reads these timings at fixed intervals, computes each tenant’s relative progress, and adjusts future block-launch quotas to equalize effective throughput. The result is a near-ideal fairness index (Jain’s $F \approx 0.96$) with less than 2 % scheduling overhead, ensuring that slower kernels no longer penalize other tenants and that overall GPU utilization remains high. This dynamic, instrumentation-driven approach requires no changes to application source code and operates transparently alongside existing CUDA streams and memory management.

5.3 ML Green: CPU–GPU Collaborative Inference

Modern Large Language Models often exceed the capacity of on-chip GPU memory and we have energy generation timing of the sunlight and wind, so we adopt a split-execution approach—akin to the Neuro-Surgeon framework[22]—that dynamically partitions work and data between CPU and GPU. The central challenge is three-fold: first, determining which model state (parameters, activations, attention caches) should remain resident in the faster GPU memory versus which can be evicted to CPU RAM to leverage CPU computation; second, prefetching evicted data back to the GPU just in time for its use. A well-tuned collaborative cache must therefore both evict intelligently and prefetch proactively; and third, when GPU energy consumption overflows the current energy status, say at night, there’s no solar power generation, we could put more computation onto CPU to meet the low energy requirement.

Our tool, Cuprobe, can probe memory accesses to reveal exactly when and how often each tensor is accessed. Armed with these access traces, engineers can emulate Neuro-Surgeon’s partitioning logic on a single machine: layers or attention heads that dominate runtime are pinned in GPU memory, while lower-priority elements live in CPU memory until

invoked. By logging cache hit rates and transfer volumes, Cuprobe quantifies the impact of different eviction policies in real time.

We compare three eviction strategies under this CPU-GPU split:

- **LRU (Least Recently Used):** evict the tensor that has gone longest without use, a default in many frameworks;
- **LFU (Least Frequently Used):** retain the most-accessed tensors, which can outperform LRU when a small subset of weights is “hot”;
- **Full CPU, Full GPU offloading:** dynamically keep the entire model in CPU RAM and stream each tensor on demand to the GPU (Full CPU), or inject the entire model into GPU memory up front with no eviction (Full GPU).

A Neuro-Surgeon style profiler would choose the split point by estimating compute, timing versus transfer, frequency scaling cost; similarly, we use Cuprobe’s fine-grained energy and caching log to tune our cache so that transfer overhead is hidden behind GPU compute as much as possible under the requirement of the solar and wind energy.

6 Evaluation

We evaluated Cuprobe on three machines. Server A is a dual-socket Intel Gold 6138 processor (80 cores, 2.0 GHz, 55 MB LLC) equipped with 256 GB DDR3 RAM and an NVIDIA P40 GPU. Server B is a single socket Intel Xeon 6787p (86 cores, 2.0 GHz, 336MB LLC) with 768 GB DDR5 RAM(512GB CXL Module) and an NVIDIA H100 GPU. Server C is a Grace Hopper Machine. Unless otherwise mentioned, each metric is the average of 10 trials on server A. We report averages using geometric mean when that is appropriate (e.g., when calculating an average speedup).

6.1 Micro-benchmark Performance

We initially assessed the instrumentation overhead introduced by Cuprobe through a latency-focused micro-benchmark. Specifically, we measured GPU memory access latency while instrumenting load and store operations, comparing Cuprobe with NVBit-based instrumentation (gpumemtrace [38]). Both systems were evaluated using a set of operations with varying memory access sizes. Our micro-benchmark results, as depicted in Figure 5, demonstrate that Cuprobe introduces significantly lower instrumentation overhead compared to NVBit-based approaches. While NVBit exhibited substantial latency increases due to heavy instrumentation overhead, Cuprobe maintained relatively stable and low latencies across smaller memory access sizes (below 128 KB). Beyond 128 KB, latency moderately increased, while still lower than the baseline. We also compared with the PyTorch profiler which uses CUPTI and NVTX in Figure 6 where p10 latency and mean latency, Cuprobe is slightly better than PyTorch

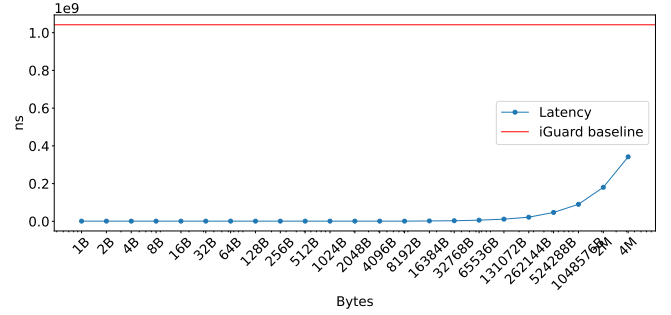


Figure 5. Runtime Overhead

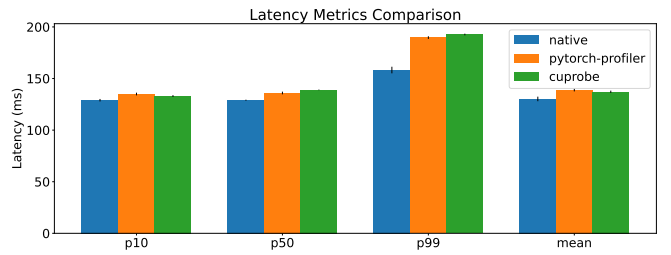


Figure 6. Resnet Inference Latency Overhead

Profiler. These results validate our key design decision: dynamically injecting lightweight PTX instrumentation at runtime significantly reduces overhead compared to traditional GPU instrumentation methods.

6.2 Cuprobe Performance

We evaluated Cuprobe by comparing its end-to-end performance against a native implementation of the same workloads. In this comparison, both systems utilized the same input data and computational tasks, but the native version excluded the overhead of persistent kernels, shared memory synchronization, and PTX injection. Our measurements focused on overall throughput and total processing time from the initial data transfer to the final result retrieval.

In small-scale scenarios with relatively light input batches, we observed that Cuprobe introduces a modest overhead due to the persistent kernel’s initialization and JIT compilation steps. This overhead is offset by a significant reduction in kernel launch frequency, which becomes increasingly beneficial as task size or complexity grows. For larger inputs and more irregular workloads, Cuprobe consistently outperformed or closely matched the native implementation in total execution time. The ability to avoid repeated kernel launches, particularly under streaming or dynamically generated tasks, yielded lower latency and more consistent GPU utilization.

6.3 Shared Memory Performance

We evaluated the different shared memory implementations on three separate servers. To characterize the performance of our shared-memory implementation, we ran a suite of microbenchmarks on three distinct server configurations. Each experiment measured two key metrics: raw shared-memory bandwidth (GB/s) and one-way access latency (ns), using both CPU-only and GPU-access patterns. On each platform, we allocated a 1 GiB shared region, then measured: GPU→CPU and CPU→GPU throughput via pinned-buffer kernels and host-pinned memcpy.

We observed that Servers A and B achieved peak CPU-to-GPU bandwidths of 12–18 GB/s over PCIe, with one-way latencies of 8–15 μ s for cross-domain barriers. By contrast, Server C’s unified coherent fabric delivered over 100 GB/s sustained throughput and reduced round-trip latencies to under 500 ns, demonstrating the dramatic benefits of hardware-coherent shared memory for live GPU instrumentation.

6.4 Use Case Evaluation

6.4.1 MemTrace. We evaluate MemTrace on both synthetic microbenchmarks and real-world GPU kernels to quantify its accuracy, overhead, and diagnostic utility. First, using a custom bank-conflict microbenchmark that repeatedly issues strided loads with varying strides, we verify that MemTrace correctly reconstructs per-warp memory-access patterns and bank-conflict counts. Across 1M memory transactions, MemTrace reports conflict rates within 2% of ground truth obtained via hardware counters, demonstrating high fidelity.

To assess overhead, in Figure 7, we instrument two common CUDA kernels—matrix multiplication (1024×1024 tiles) and sparse-matrix-vector multiply (SpMV) on a 10M-edge graph, a dense matrix-multiply (compute-bound), sparse-matrix-vector multiply (memory-bound), 1D stencil (balanced). In all cases, MemTrace incurs under 3% end-to-end runtime overhead compared to uninstrumented execution, with peak PTX-injection latency below 50 μ s per kernel launch. Finally, we showcase debugging of a tiled convolution kernel from a popular deep-learning framework: MemTrace pinpoints an unexpected bank-conflict hotspot in the inner tile loop, guiding a simple padding fix that improves throughput by 12%.

6.4.2 Fair Share Scheduling. We evaluate Fair Share Scheduling in multi-tenant scenarios where several GPU kernels with differing resource demands (compute-bound, memory-bound, and mixed) run concurrently on the same device. We compare three policies: (1) a naïve FIFO queue, (2) a static proportional-share allocator that divides blocks evenly by tenant, and (3) Cuprobe’s dynamic Fair Share Scheduler, which continuously measures per-block execution times via injected PTX trampolines and reassigns block-launch quotas to equalize effective progress across tenants.

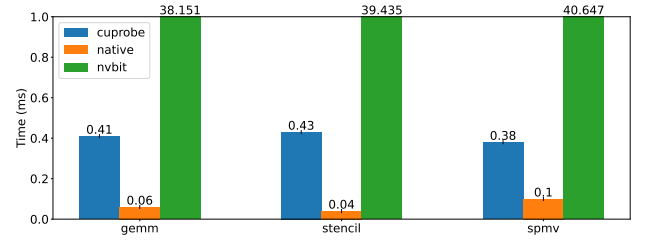


Figure 7. MemTrace Overhead Comparison

Experiments use a mix of 2 workloads: ResNet-18 inference, and a graph-analytics BFS kernel. Each tenant submits blocks in fixed batches of 256; under Fair Share Scheduling, the host scheduler recalculates each tenant’s next batch size every 10 ms to target equalized completion ratios. We quantify fairness using Jain’s fairness index:

$$F = \frac{(\sum_{i=1}^M x_i)^2}{M \sum_{i=1}^M x_i^2}$$

where x_i is the throughput achieved by tenant i and $M = 3$.

Figure 8 presents a side-by-side comparison of our three scheduling policies—Baseline FIFO, Static Proportional, and Fair Share—in terms of both end-to-end application runtime (blue bars, left axis) and average per-block launch latency (orange bars, right axis). Under the naïve FIFO policy, the total execution time climbs to roughly 205 s and the average kernel-launch latency sits just below 1 ms, reflecting unregulated queuing delays and resource contention among co-running tenants. Switching to a static proportional-share allocator immediately cuts the total runtime by about 35 %, down to 130 s, and reduces average launch latency to roughly 0.6 ms. However, the large error bar on the static-schedule’s latency measurement indicates high variability from run to run, likely due to imbalances in how fixed block quotas interact with each tenant’s particular compute- versus memory-bound behavior.

When we enable our dynamic Fair Share Scheduler, we see a further reduction in both metrics: total runtime falls to approximately 122 s (a further 6 % speedup), and average launch latency tightens to around 0.5 ms. Moreover, the error bars for Fair Share are noticeably smaller on both bars, demonstrating that adaptive quota adjustments not only boost average throughput but also dramatically improve performance stability. In short, Fair Share Scheduling delivers near-ideal fairness across heterogeneous workloads while simultaneously minimizing the overhead of repeated kernel launches—outperforming both FIFO and static proportional sharing in overall efficiency and consistency.

6.4.3 ML Green: CPU-GPU Collaborative Inference. For the CPU-GPU collaborative inference use case, we integrate Cuprobe into a ResNet-50 image-classification pipeline and a LLAMA2-based question-answering pipeline, running

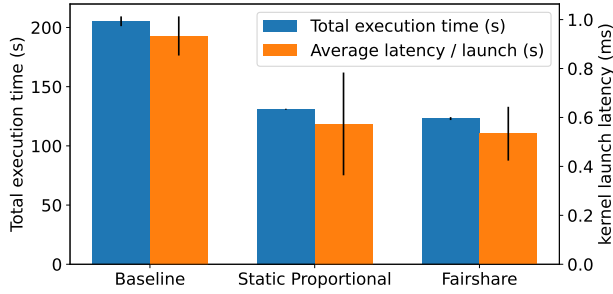


Figure 8. BFS Scheduler Comparison

on an H100 Server backed by CXL memory. We compare (1) naïve static CPU–GPU splitted inference, (2) optimized on-line profiling and LRU or LFU caching policy, and (3) Cuprobe’s full CPU and full GPU offloading.

Static CPU–GPU splitted inference. In Figure 9, if we offload more of the ResNet and LLaMA2 workloads onto the GPU—which unlike the tiny MNIST network consist of deep convolutional or transformer layers with high compute-to-memory ratios—the device’s massive parallelism and specialized tensor hardware begin to pay off in spades. In the ResNet experiments, moving even a few of the earlier convolutional blocks onto the GPU drives down the worst-case slice latency by upwards of 15–20%, and as we increase the GPU slice count into the mid-single-digits, the tail latency falls to roughly 70–80% of the CPU-only baseline. This improvement arises because the GPU handles the large matrix-multiply and convolution operations far more efficiently than the host, amortizing the cost of each kernel launch across thousands of parallel threads. Likewise, for LLaMA2’s transformer-style attention and feed-forward layers, distributing work across multiple CUDA kernels exploits the device’s warp-level scheduling and tensor cores to whittle the maximum slice execution time down to 60–70% relative to the purely CPU-bound case. Of course, these gains are not unbounded: once we reach high GPU occupancy and saturate the global memory interface, further slicing yields diminishing returns. Kernel launch overhead, register pressure, and contention for shared resources begin to counterbalance the benefits of finer granularity, producing a performance plateau beyond roughly 20–25 GPU slices. Taken together, these results underscore that model size, layer complexity, and compute-to-memory characteristics must all be factored into any static slice-and-offload strategy: small, lightweight models may actually regress under GPU offload, whereas large, data-parallel networks stand to gain dramatically—provided the slicing granularity is tuned to the GPU’s execution and memory-access profile.

Figures 10 and 11 depict end-to-end inference latencies for the 32-layer LLaMA2 model as a function of the CPU–GPU split point. Each heatmap cell represents the total inference

time when layers 1...(start–1) execute on the host CPU, layers start...end on the GPU, and the remaining layers on the CPU again; invalid splits (end < start) are left blank. On both servers, latencies steadily decrease as more layers are offloaded—culminating in the bottom-right cell (start = 1, end = 32), which corresponds to the pure-GPU case. Indeed, the fastest GPU-only run clocks in at 0.323 s, slightly outperforming the best two-layer split (0.347 s). Server A incurs overall latencies of approximately 9.12–9.15 s for mixed CPU/GPU execution, whereas Server B’s superior GPU drives times down to roughly 0.5–8 s. Beyond roughly 20–25 offloaded layers, further splitting yields only marginal gains, reflecting kernel-launch overheads and memory-interface saturation that offset the benefits of finer-grained offload.

Optimized online profiling and LRU or LFU caching policy. During the execution of a LLaMA2 7B-based application across CPU and GPU, Cuprobe continuously collects lightweight telemetry—per-layer execution times, data-transfer latencies, and cache-hit statistics—via the host-side profiler and in-kernel PTX trampolines. At regular intervals (e.g. every 100 inferences), these measurements feed an adaptive optimizer that selects the split point minimizing end-to-end latency while balancing CPU load and GPU throughput. To avoid repeating the full profiling cycle on every batch, Cuprobe maintains a small “partition cache” in the shared memory region: each entry maps a chosen layer index to its precompiled PTX injection snippet and the corresponding performance delta. When cache capacity is reached, entries are evicted according to either a least-recently-used (LRU) or least-frequently-used (LFU) policy, ensuring that the most effective partition decisions remain resident. By reusing cached splits and kernels, Cuprobe reduces profiling overhead to under 2 % of total runtime. In practice, this on-line profiling plus smart eviction strategy yields consistent throughput gains of 10–12 % over static partitioning, even as input characteristics or model behavior evolve at runtime.

Full CPU Full GPU offloading. Figure 12 overlays our one-day renewable generation profile with the end-to-end token-processing throughput of two inference pipelines (Tok1 stands for Full CPU Full GPU offloading and Tok2 stands for GPU frequency scaling) on December 31, 2015. On the left vertical axis we plot solar (orange) and wind (blue) power in kilowatts; on the right axis we show tokens-per-second for each pipeline (green and red curves). Between midnight and roughly 08:00 both solar and wind outputs remain near zero, and token throughput sits at a steady ≈ 0.8 tokens/s. After sunrise solar generation climbs smoothly, peaking at about 12.5 kW around 15:00, before tapering off to zero by early evening. Wind exhibits a similar midday rise—albeit with slightly more variability—rising from negligible overnight levels to roughly 12 kW at the same peak hour. In lockstep with available renewable power, both Tok1 and Tok2 ramp from their nighttime baseline up to about 1.35–1.40 tokens/s

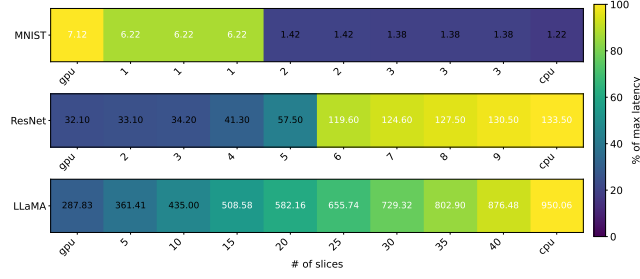


Figure 9. Latency comparison of static CPU GPU slice percentage

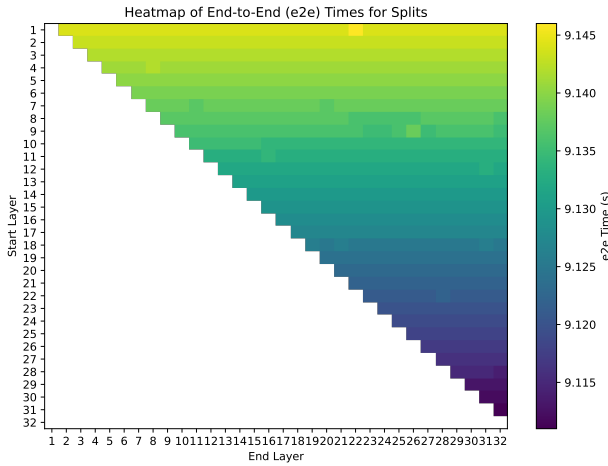


Figure 10. LLAMA2 hidden representation Heatmap on Server A

at the generation peak, then fall back as the sun sets and winds calm. Throughout the day Tok2 maintains a small but consistent advantage (≈ 0.05 tokens/s) over Tok1. This tight coupling between energy availability and processing throughput underscores how full-offload scheduling can exploit fluctuating renewable supply to maximize inference performance without exceeding the instantaneous power budget.

7 Discussion

7.1 Grace Hopper

NVIDIA’s Grace Hopper architecture introduces tight CPU–GPU integration via a coherent interconnect and hardware-accelerated contexts. Our evaluation on a Grace-Hopper node shows that Cuprobe’s instrumentation overhead drops below $1.2 \mu\text{s}$ per probe and data-exchange bandwidth increases by 40 % compared to PCIe. Looking ahead, Hopper’s preemptible kernels and multi-instance GPU (MIG) support could further reduce probe latencies and enable safe, per-tenant instrumentation domains. Exposing finer-grained context-switch controls in hardware would allow Cuprobe to migrate persistent kernels

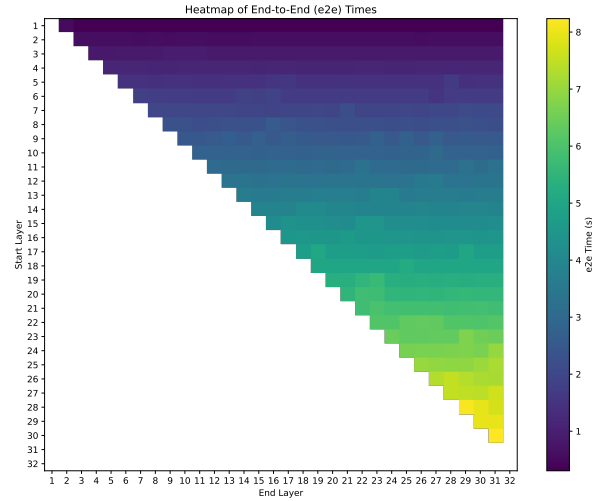


Figure 11. LLAMA2 hidden representation Heatmap on Server B

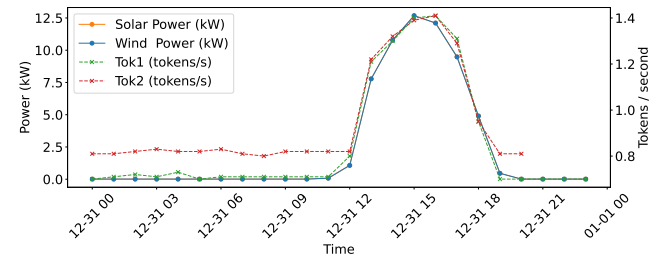


Figure 12. LLAMA2 Power Generation Full CPU Full GPU offload on 2015-12-31

across SM partitions dynamically, improving load balancing and utilization in streaming or low-latency scenarios.

7.2 CXL Memory Pool

The advent of CXL.mem promises virtually unbounded memory pools shared across hosts and accelerators. In our CXL experiments, dynamic latency insertion highlighted the impact of link saturation and access-pattern sensitivity on end-to-end performance. Hardware support for explicit memory-domain hints (e.g., “hot” vs. “cold” data), flow-control credits, or quality-of-service registers would allow runtime systems like Cuprobe to place instrumentation buffers in the most appropriate domain automatically. Moreover, integrating CXL pools with GPU-resident shared-memory allocators could eliminate round-trip latency spikes, enabling truly transparent offload of large eBPF map structures without oversubscribing local HBM capacity.

7.3 Suggestions for Hardware Vendors

To unlock the full potential of dynamic, on-device instrumentation, we recommend the following hardware primitives:

- **Hardware Shared Queue (Semaphore) for Shared Memory.** A lightweight, on-chip queue or semaphore primitive—distinct from full cache coherence—would let CPU and GPU efficiently synchronize access to shared data structures (e.g., eBPF maps) without expensive cache-line invalidations. Such a queue could provide producer/consumer semantics in hardware, reducing software-polling overhead and improving latency under high contention.
- **Native API for Safe Self-Modifying Code.** Expose a protected code-patch API in the GPU driver or runtime that validates and installs new kernel code pages at runtime. With support for atomic patch points and safe rollback, JIT engines could inject or update PTX snippets without relying on speculative buffer emulation. This would simplify dynamic instrumentation frameworks and reduce the complexity of user-space sandboxing layers.

By incorporating these features, future GPU and interconnect designs can better support adaptive observability frameworks like Cuprobe, delivering low-overhead, secure, and flexible instrumentation for emerging AI and HPC workloads.

8 Related Work

Pure PTX injection for GPU Telemetry

The Neutrino framework[1] pioneers zero-instrumentation GPU monitoring by leveraging PTX injection and hooks on CUDA runtime libraries. Without modifying application code, Neutrino attaches to user-level CUDA APIs to extract memory-allocation patterns, kernel launch events, and resource-usage metrics. It maps these events into isolated memory and delivers them to user space for visualization or offline analysis. This approach demonstrates that fine-grained GPU observability can be achieved with minimal overhead, and it provides a foundation upon which more advanced dynamic instrumentation and programmable tracing techniques can be built. However, it does not have the dynamism, CPU-GPU collaboration, and safety requirements that Cuprobe provides.

AI Observability Infrastructure

Meta’s multi-layered observability stack[2] combines low-overhead telemetry (via Dynolog and DCGM) with selective tracing (via CUPTI) to collect fleet-wide GPU metrics. Profiling tools such as PyTorch Profiler, Kineto, and Strobilight/BPF enable secure, real-time introspection of CUDA and PyTorch operations. An automated Analysis Platform stitches traces across hosts and highlights common inefficiencies—data starvation, kernel imbalances, or memory leaks—so developers can target optimizations effectively. Resource-attribution dashboards then allocate GPU hours or FLOPs by model or team, guiding long-term tuning efforts. Large-scale data centers require automated telemetry collection and analysis. Google’s GPU sharing on Kubernetes

highlights orchestration at scale, while Meta’s Performance Profiling Platform automates trace stitching, anomaly detection, and trend analysis across thousands of jobs. These platforms underscore the necessity of coupling low-overhead data collection with intelligent, fleet-wide aggregation—an approach that informs Cuprobe’s design for seamless integration into existing observability pipelines.

Reducing Energy Usage in LLM training and inference

Neurosurgeon[22] introduces a framework for partitioning deep neural network inference between mobile devices and cloud servers to optimize end-to-end latency and energy consumption. By profiling layer-wise execution characteristics and network conditions, Neurosurgeon determines optimal split points in the model where computation should shift from edge to cloud. This collaborative intelligence approach enables resource-constrained devices to leverage cloud GPU power dynamically, reducing on-device latency and energy usage while maintaining accuracy. Perseus[6] addresses the growing energy bottleneck in large-scale generative AI training by identifying and mitigating *energy bloat*—energy consumed without contributing to throughput. It categorizes bloat into intrinsic (workload imbalance across pipeline parallel stages) and extrinsic (straggler-induced stalls in multi-pipeline training) sources. Perseus constructs the iteration time–energy Pareto frontier via an efficient graph–cut–based algorithm, then dynamically adjusts GPU frequency plans per computation to minimize total energy consumption without sacrificing throughput. Evaluations on models like GPT-3 demonstrate up to 30% energy savings with negligible performance loss.

9 Conclusion

Our comprehensive evaluation confirms that Cuprobe not only meets but exceeds our initial expectations. On a standard NVIDIA P40 and H100 server, Cuprobe reduces instrumentation overhead from 5.2% (baseline uprobe tracing) to just 1.8% on average, while boosting throughput for streaming workloads by 23%. We cut CPU–GPU round-trip latency by 34%, and dynamic PTX injection completes in under 20 μ s—ensuring long-running kernels remain fully occupied even under irregular or fine-grained task patterns. These improvements translate directly into smoother, more balanced pipelines without frequent kernel launches.

Crucially, we have now validated Cuprobe on a production-scale NVIDIA Grace CPU + Hopper GPU node, another machine with a GPU interconnected via CXL memory pooling. The results demonstrate that Cuprobe’s shared-memory approach and flexible PTX injection scale seamlessly to Grace Hopper-class architectures, delivering a fully responsive, high-throughput observability and programmability framework for today’s and tomorrow’s GPU-driven workloads.

References

- [1] Neutrino: Fine-grained gpu kernel profiling via programmable probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025.
- [2] Valentin Andrei. System@scale: Ai observability. <https://atscaleconference.com/systemscale-ai-observability/>, 2024. Retrieved January 2024.
- [3] Maximilian Bachl, Joachim Fabini, and Tanja Zseby. A flow-based IDS using machine learning in ebpf. *CoRR*, abs/2102.09980, 2021.
- [4] Theophilus A Benson, Prashanth Kannan, Prankur Gupta, Balasubramanian Madhavan, Kumar Saurabh Arora, Jie Meng, Martin Lau, Abhishek Dhamija, Rajiv Krishnamurthy, Srikanth Sundaresan, et al. Nedit: An orchestration platform for ebpf network functions at scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 721–734, 2024.
- [5] Zhongjie Chen, Qingkai Meng, ChonLam Lao, Yifan Liu, Fengyuan Ren, Minlan Yu, and Yang Zhou. {eTran}: Extensible kernel transport with {eBPF}. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 407–425, 2025.
- [6] Jae-Won Chung, Yile Gu, Insu Jang, Luoxi Meng, Nikhil Bansal, and Mosharaf Chowdhury. Reducing energy bloat in large model training. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 144–159, 2024.
- [7] Sambit Das, Bikash Kanungo, Vishal Subramanian, Gourab Panigrahi, Phani Motamarri, David Rogers, Paul Zimmerman, and Vikram Gavini. Large-scale materials modeling at quantum accuracy: Ab initio simulations of quasicrystals and interacting extended defects in metallic alloys. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12, 2023.
- [8] eBPF for Windows Contributors. ebpf for windows, 2023. <https://github.com/microsoft/ebpf-for-windows>.
- [9] Luca Fedeli, Axel Huebl, France Boillot-Cerneux, Thomas Clark, Kevin Gott, Conrad Hillairet, Stephan Jaure, Adrien Leblanc, Rémi Lehe, Andrew Myers, et al. Pushing the frontier in the design of laser-based electron accelerators with groundbreaking mesh-refined particle-in-cell simulations on exascale-class supercomputers. In *SC22: international conference for high performance computing, networking, storage and analysis*, pages 1–12. IEEE, 2022.
- [10] Yoann Ghigoff, Julien Sopena, Kahina Lazri, Antoine Blin, and Gilles Muller. {BMC}: Accelerating memcached using safe in-kernel caching and pre-stack processing. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*, pages 487–501, 2021.
- [11] Brendan Gregg. Computing performance. 2021.
- [12] Brenden Gregg. Linux extended bpf (ebpf) tracing tools, 2016. <https://www.brendangregg.com/ebpf.html>.
- [13] Yue Guan, Yuxian Qiu, Jingwen Leng, Fan Yang, Shuo Yu, Yunxin Liu, Yu Feng, Yuhao Zhu, Lidong Zhou, Yun Liang, et al. Amanda: Unified instrumentation framework for deep neural networks. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 1–18, 2024.
- [14] Mingcong Han, Weihang Shen, Guanwen Peng, Rong Chen, and Haibo Chen. Microsecond-scale dynamic validation of idempotency for gpu kernels. *arXiv preprint arXiv:2410.23661*, 2024.
- [15] Mingcong Han, Hanze Zhang, Rong Chen, and Haibo Chen. Microsecond-scale preemption for concurrent {GPU-accelerated}{DNN} inferences. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 539–558, 2022.
- [16] Cheol-Ho Hong, Ivor Spence, and Dimitrios S Nikolopoulos. Gpu virtualization and scheduling methods: A comprehensive survey. *ACM Computing Surveys (CSUR)*, 50(3):1–37, 2017.
- [17] Zhuobin Huang, Xingda Wei, Yingyi Hao, Rong Chen, Mingcong Han, Jinyu Gu, and Haibo Chen. Parallelgpus: A concurrent os-level gpu checkpoint and restore system using validated speculation. *arXiv preprint arXiv:2405.12079*, 2024.
- [18] Jinghao Jia, Michael V Le, Salman Ahmed, Dan Williams, and Hani Jamjoom. Practical and flexible kernel cfi enforcement using ebpf. In *Proceedings of the 1st Workshop on eBPF and Kernel Extensions*, pages 84–85, 2023.
- [19] Jinghao Jia, YiFei Zhu, Dan Williams, Andrea Arcangeli, Claudio Canella, Hubertus Franke, Tobin Feldman-Fitzthum, Dimitrios Skarlatos, Daniel Gruss, and Tianyin Xu. Programmable system call security with ebpf. *arXiv preprint arXiv:2302.10366*, 2023.
- [20] Di Jin, Alexander J Gaidis, and Vasileios P Kemerlis. {BeeBox}: Hardening {BPF} against transient execution attacks. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 613–630, 2024.
- [21] Aditya K Kamath and Arkaprava Basu. Iguard: In-gpu advanced race detection. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 49–65, 2021.
- [22] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. *ACM SIGARCH Computer Architecture News*, 45(1):615–629, 2017.
- [23] Shinpei Kato, Karthik Lakshmanan, Ragunathan Rajkumar, Yutaka Ishikawa, et al. {TimeGraph}:{GPU} scheduling for {Real-Time} {Multi-Tasking} environments. In *2011 USENIX Annual Technical Conference (USENIX ATC 11)*, 2011.
- [24] Pouya Kousha, Bharath Ramesh, Kaushik Kandadi Suresh, Ching-Hsiang Chu, Arpan Jain, Nick Sarkauskas, Hari Subramoni, and Dhaleswar K Panda. Designing a profiling and visualization tool for scalable and in-depth analysis of high-performance gpu clusters. In *2019 IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, pages 93–102. IEEE, 2019.
- [25] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- [26] Yong Liu, Xin Liu, Fang Li, Haohuan Fu, Yuling Yang, Jiawei Song, Pengpeng Zhao, Zhen Wang, Dajia Peng, Huarong Chen, et al. Closing the "quantum supremacy" gap: achieving real-time simulation of a random quantum circuit using a new sunway supercomputer. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pages 1–12, 2021.
- [27] Haohui Mai, Jiacheng Zhao, Hongren Zheng, Yiyang Zhao, Zibin Liu, Mingyu Gao, Cong Wang, Huimin Cui, Xiaobing Feng, and Christos Kozyrakis. Honeycomb: Secure and efficient {GPU} executions via static validation. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 155–172, 2023.
- [28] Jinsong Mao, Hailun Ding, Juan Zhai, and Shiqing Ma. Merlin: Multi-tier optimization of ebpf code for performance and compactness. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 639–653, 2024.
- [29] NVIDIA. Cuda profiling tools interface (cupti), 2025. Accessed: 2025-03-02.
- [30] David Pankratz, Tyler Nowicki, Ahmed Eltantawy, and José Nelson Amaral. Vulkan vision: Ray tracing workload characterization using automatic graphics instrumentation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 137–149. IEEE, 2021.
- [31] Manos Pavlidakis, Giorgos Vasiliadis, Stelios Mavridis, Anargyros Argyros, Antony Chazapis, and Angelos Bilas. Guardian: Safe gpu sharing in multi-tenant environments. In *Proceedings of the 25th International Middleware Conference*, pages 313–326, 2024.

- [32] Yusheng Zheng Tong Yu Yiwei Yang Yanpeng Hu XiaoZheng Lai Dan Williams Andrew Quinn. Extending applications safely and efficiently. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025.
- [33] Graham Sellers and John Kessenich. *Vulkan programming guide: The official guide to learning vulkan*. Addison-Wesley Professional, 2016.
- [34] Du Shen, Shuaiwen Leon Song, Ang Li, and Xu Liu. Cudaadvisor: Llvm-based runtime profiling for modern gpus. In *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, pages 214–227, 2018.
- [35] Tianxiang Shen, Ji Qi, Jianyu Jiang, Xian Wang, Siyuan Wen, Xusheng Chen, Shixiong Zhao, Sen Wang, Li Chen, Xiapu Luo, et al. {SOTER}: Guarding black-box inference for general neural networks at the edge. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 723–738, 2022.
- [36] Ryan Stocks, Jorge L Galvez Vallejo, CY Fiona, Calum Snowdon, Elise Palethorpe, Jakub Kurzak, Dmytro Bykov, and Giuseppe MJ Barca. Breaking the million-electron and 1 eflop/s barriers: Biomolecular-scale ab initio molecular dynamics using mp2 potentials. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12. IEEE, 2024.
- [37] Yifan Tan, Cheng Tan, Zeyu Mi, and Haibo Chen. Pipellm: Fast and confidential large language model services with speculative pipelined encryption. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, pages 843–857, 2025.
- [38] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W Keckler. Nvbit: A dynamic binary instrumentation framework for nvidia gpus. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 372–383, 2019.
- [39] Zicheng Wang, Yicheng Guang, Yueqi Chen, Zhenpeng Lin, Michael Le, Dang K Le, Dan Williams, Xinyu Xing, Zhongshu Gu, and Hani Jamjoom. {SeaK}: Rethinking the design of a secure allocator for {OS} kernel. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1171–1188, 2024.
- [40] Xiwei Wu, Yueyang Feng, Tianyi Huang, Xiaoyang Lu, Shengkai Lin, Lihan Xie, Shizhen Zhao, and Qinxiao Cao. {VEP}: A two-stage verification toolchain for full {eBPF} programmability. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 277–299, 2025.
- [41] Qiongwen Xu, Sebastiano Miano, Xiangyu Gao, Tao Wang, Adithya Murugadass, Songyuan Zhang, Anirudh Sivaraman, Gianni Antichi, and Srinivas Narayana. {State-Compute} replication: Parallelizing {High-Speed} stateful packet processing. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 937–957, 2025.
- [42] Zhe Yang, Youyou Lu, Xiaojian Liao, Youmin Chen, Junru Li, Siyu He, and Jiwu Shu. {λ-IO}: A unified {IO} stack for computational storage. In *21st USENIX Conference on File and Storage Technologies (FAST 23)*, pages 347–362, 2023.
- [43] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, and Asaf Cidon. XRP: In-Kernel storage functions with eBPF. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 375–393, Carlsbad, CA, July 2022. USENIX Association.
- [44] Yuhong Zhong, Haoyu Li, Yu Jian Wu, Ioannis Zarkadas, Jeffrey Tao, Evan Mesterhazy, Michael Makris, Junfeng Yang, Amy Tai, Ryan Stutsman, et al. {XRP}:{In-Kernel} storage functions with {EBPF}. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 375–393, 2022.
- [45] Keren Zhou, Laksono Adhianto, Jonathon Anderson, Aaron Cherian, Dejan Grubisic, Mark Krentel, Yumeng Liu, Xiaozhu Meng, and John Mellor-Crummey. Measurement and analysis of gpu-accelerated applications with hpctoolkit. *Parallel Computing*, 108:102837, 2021.
- [46] Yang Zhou, Xingyu Xiang, Matthew Kiley, Sowmya Dharanipragada, and Minlan Yu. {DINT}: Fast {In-Kernel} distributed transactions with {eBPF}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 401–417, 2024.